Chapter 14

CodeAgent: An LLM-Powered C++ Coding Copilot with CLI and Browser GUI

Lalitadithya Kariyam (251130)
Samuel John (251118)

14.1 Introduction and Problem Statement

Large language models (LLMs) can write and improve source code, but using them from a chat website is awkward for real development work. The developer has to:

- copy code in and out by hand;
- check whether the answer is complete;
- avoid pasting secrets such as API keys;
- make sure nothing overwrites their files by accident.

There is no review step between *"the model suggested a change"* and *"my file was changed"*.

CodeAgent solves this. It is a C++17 application that connects Google's Gemini model to a developer's local workflow through two front ends that share one backend:

1. **A command-line interface (CLI):**
   - `codeagent generate "<instruction>"` generates code and offers to save it.
   - `codeagent refactor <file> "<instruction>"` reads a file, asks Gemini for an improved version, shows a unified diff, and writes the file only after explicit approval.
2. **A browser-based GUI** (`codeagent gui`). The same executable starts a local, loopback-only HTTP server. That server serves an editor page and a JSON API offering generation, refactoring proposals, diff review, approve and reject, and safe file saving.

End-user view

The user types an instruction in plain English, either in the terminal or in the browser chat panel. CodeAgent builds a carefully worded prompt, sends it over HTTPS to Gemini, checks that the reply is complete and usable, and displays the code. For a refactoring, the user always sees what will change before anything happens.

Software requirements

R1 Generate code from a natural-language instruction.

R2 Refactor an existing text file and present the change as a diff.

R3 Never modify a file without explicit user consent; approval and saving are separate actions.

R4 Read and write files safely: atomic replacement, no silent overwrite, no symbolic-link targets, and no secret (`.env`) files.

R5 Keep the API key secret: never in a URL, never in logs, never sent to the browser.

R6 Report every failure (network, quota, invalid key, blocked or incomplete response, file error) with a clear, actionable message instead of crashing.

R7 Offer the same core features from both a CLI and a GUI without duplicating backend logic.

R8 Be testable without network access or a real Gemini account.

14.2 Scope and Feature Set

- **AI code generation (CLI and GUI):** instruction + language → Gemini → validated source code. In the CLI the result can be saved to a user-chosen path. In the GUI it is accepted into the editor and then saved with *Save As*.
- **AI refactoring with review:** the CLI shows a unified diff and asks `Apply these changes? [y/N]`. The GUI creates a server-side *proposal* (`PENDING`) that must be reviewed as a diff, then approved or rejected. Approval only updates the editor; saving is a separate, explicit step.
- **Safe file I/O and persistent storage:** UTF-8 text files of any extension are read and written atomically (temporary file + rename); binary content and environment (`.env`) files are refused; overwriting needs confirmation; external modification is detected with a content revision hash; "always allow" permissions persist in `.codeagent/permissions.txt`; configuration comes from layered `.env` files.
- **Robust Gemini integration:** HTTPS through libcurl with TLS verification, timeouts and a 2 MiB response limit; strict JSON validation; finish-reason checking (a refactoring must be explicitly complete); removal of Markdown code fences; optional display of the model's "thoughts".

- **Typed error handling:** a polymorphic exception hierarchy maps each failure to a user message, a machine-readable code and an HTTP status, with automatic secret redaction.
- **Secure local web server:** bound to `127.0.0.1` only, with a one-time bootstrap code exchanged for a 256-bit session token, Host and Origin header checks, a strict Content Security Policy, a 1 MiB request limit and exact-shape JSON validation.
- **Native dialogs:** OS file and folder pickers (zenity on Linux; a Windows dialog through PowerShell under WSL), started without a shell.
- **Automated tests:** 214 GoogleTest cases (C++) and 79 Node.js tests (frontend), none of which need the internet.

Out of scope (current limitations)

- Linux and WSL2 are the supported platforms; native Windows and macOS builds were not verified.
- The GUI editor is a plain text area (no syntax highlighting).
- Proposals are kept in memory and are lost when the server stops.
- Generated code is not compiled or executed by CodeAgent.
- The CLI `generate` command always targets C++; the GUI lets the user choose the language.

14.3 Evaluation Criteria Mapping

Table 14.1 shows where each evaluation criterion is addressed in this report and in the repository.

Criterion	Report sections	Repository evidence
OOP Architecture & Design (25)	14.6 Object-Oriented Architecture, 14.7 UML	`include/*.hpp`, `UML-diagrams.md`
Advanced C++ Mechanics (20)	14.8 Advanced C++ Mechanics	`CodeAgentError.*`, `LLMClient.cpp`, `FileManager.cpp`, `GuiHttpServer.cpp`
Frontend & Backend Integration (20)	14.4 Technology Stack, 14.9 Integration, 14.9.4 Persistent Storage	`src/`, `frontend/`, `docs/gui-api-contract.md`
Version Control (10)	14.12 Version Control	61 commits, 6 branches, 3 merge commits
Documentation & Analysis (10)	This chapter, 14.10 Testing	`README.md`, `docs/`
Demonstration & Viva (15)	14.13 Results, 14.14 Demo plan	`docs/demo-and-viva.md`, `scripts/smoke-test.sh`

Table 14.1: Evaluation criteria and supporting evidence

14.4 Technology Stack

Table 14.2 lists the technologies used in each layer of CodeAgent. All business logic is written in C++; the browser frontend uses plain web technologies with no framework or build step.

Layer	**Technology**	**Used for**
Language	C++17	All backend logic and the CLI
Build	CMake 3.15+, FetchContent	Libraries, the executable and test targets
AI model	Google Gemini API	Code generation and refactoring
HTTP client	libcurl (HTTPS, TLS verification)	Outbound requests to Gemini
JSON	nlohmann/json	Gemini requests and responses, API payloads
HTTP server	cpp-httplib 0.58	Local JSON API on `127.0.0.1`
Frontend	HTML, CSS, JavaScript (ES modules)	Editor, chat and diff review in the browser
Testing	GoogleTest 1.14 with CTest; `node:test`	214 C++ tests and 79 frontend tests
Platform	Linux / WSL2; zenity or PowerShell	Runtime and native file dialogs
Version control	Git and GitHub	Feature branches, 61 commits by both members

Table 14.2: Technology stack

Why these choices.

- **C++17** provides `std::filesystem`, `std::optional` and the object-oriented features the project is built on, and keeps the API key and all file access inside the native process.
- **libcurl and cpp-httplib play opposite roles**: libcurl is the client that sends requests out to Gemini, while cpp-httplib is the server that receives requests from the browser.
- **No frontend framework**: the C++ executable serves the static HTML, CSS and JavaScript files itself, so the GUI needs no separate process or build tool.
- **Pinned dependencies**: GoogleTest v1.14.0 and cpp-httplib v0.58.0 are fetched at fixed versions, so every build uses the same code.

14.5 System Architecture Overview

CodeAgent is a **single executable** built in three layers:

Presentation layer (front ends). `CLIController` (terminal) and the browser application in `frontend/` (HTML, CSS and ES-module JavaScript). The browser never talks to Gemini and never touches the disk; it only calls the local API.

Application / service layer. `GuiHttpServer` exposes the backend over HTTP and owns proposal state. It does *not* call Gemini itself: it receives *generation* and *refactoring handlers* (function objects) from `CLIController`, so the Gemini logic exists only once.

Core / domain layer. `PromptBuilder` (prompt text), `LLMClient` (HTTPS to Gemini), `ResponseParser` (JSON interpretation), `FileManager` (CLI file I/O with consent), `makeUnifiedDiff` (line diff), `EnvLoader` (configuration), `CodeAgentError` (failure model) and `NativeDialogProvider` (OS dialogs).

```
Terminal user Browser user (frontend/js/app.js, api.js)
     | | HTTP + JSON + X-CodeAgent-Token
     v v 127.0.0.1:8080
main.cpp -> CLIController --- "gui" ---> GuiHttpServer (cpp-httplib, pimpl)
     | | | | | |
     | | +--> FileManager (terminal) | | +--> NativeDialogProvider
     | +------> makeUnifiedDiff <-----------+ +------> GUI safe file I/O
     | |
     +--> PromptBuilder <-- handlers ----------+
             |
             v
         LLMClient --> ResponseParser
             | (EnvLoader: GEMINI_API_KEY)
             v HTTPS (libcurl)
         Google Gemini API
```

Listing 14.1: Layered architecture of CodeAgent

14.6 Object-Oriented Architecture

The design applies the core OOP principles throughout: each class has a single responsibility, data is hidden behind small public interfaces, related failures are modelled by inheritance, behaviour varies through virtual functions, and objects are composed and injected rather than hard-wired.

14.6.1 Class Design and Associations

Table 14.3 lists the primary classes and the responsibility of each.

Inheritance.

- A three-level hierarchy of exceptions derives from `std::runtime_error` (Listing 14.2).
- `SystemNativeDialogProvider` inherits from (implements) the abstract interface `NativeDialogProvider`.
- In the tests, `ScriptedNativeDialogProvider` implements the same interface.

Class	Responsibility	Files
`CLIController`	Parses commands, runs the CLI workflows, starts the GUI, and is the CLI error boundary	`CLIController.hpp/.cpp`
`PromptBuilder`	Builds deterministic generation and refactoring prompts; validates inputs	`PromptBuilder.hpp/.cpp`
`LLMClient`	Sends prompts to Gemini over HTTPS; maps failures to typed exceptions	`LLMClient.hpp/.cpp`
`ResponseParser`	Validates Gemini JSON and extracts text and thoughts; classifies finish reasons	`ResponseParser.hpp/.cpp`
`FileManager`	Terminal-mediated reading and writing with permission prompts and atomic writes	`FileManager.hpp/.cpp`
`GuiHttpServer` (+ `Impl`)	Local HTTP API, session security, proposal store, GUI file I/O	`GuiHttpServer.hpp/.cpp`
`NativeDialogProvider` (abstract) and `SystemNativeDialogProvider`	Interface and implementation for OS file and folder pickers	`NativeDialog.hpp/.cpp`
`CodeAgentError` (abstract) + 12 subclasses	Typed, safe failure model with `code()` and `httpStatus()`	`CodeAgentError.hpp/.cpp`
Data types: `ParsedResponse`, `ProviderErrorDetails`, `GenerationResult`, `NativeDialogResult`, `PendingProposal`	Plain structures that carry results between classes	various

Table 14.3: Primary classes and responsibilities

```
std::runtime_error
 +-- CodeAgentError <<abstract>> code(), httpStatus() pure virtual
     +-- ConfigurationError CONFIGURATION_ERROR 503
     +-- ProviderError PROVIDER_ERROR 502
     | +-- AuthenticationError PROVIDER_AUTH_FAILED 502
     | +-- RateLimitError PROVIDER_RATE_LIMITED /
     | | PROVIDER_QUOTA_EXHAUSTED 429
     | +-- ProviderUnavailableError PROVIDER_UNAVAILABLE 503
     | +-- NetworkError PROVIDER_UNREACHABLE 502
     | | +-- TimeoutError PROVIDER_TIMEOUT 504
     | +-- ProviderResponseError PROVIDER_RESPONSE_* 502
     +-- FileError READ_FAILED / SAVE_FAILED 500
         +-- FilePermissionError PERMISSION_DENIED 403
         +-- FileNotFoundError FILE_NOT_FOUND 404
         +-- FileConflictError FILE_CHANGED_ON_DISK 409
```

Listing 14.2: Exception inheritance hierarchy (is-a relationships)

Composition (strong ownership).

- `GuiHttpServer` *owns* its `Impl` through `std::unique_ptr<Impl>`.
- `Impl` owns an `httplib::Server`, the listener `std::thread`, the proposal map (`std::map<std::string, PendingProposal>`), the LRU list of opened documents, and the session secrets.

These parts are created and destroyed together with their owner.

Aggregation (shared, non-owning or shared ownership).

- `GuiHttpServer` holds a `std::shared_ptr<NativeDialogProvider>` that can be shared with, and outlive, the server (a test keeps its own reference).
- `CLIController` and `FileManager` hold *references* to streams (`std::istream&`, `std::ostream&`) that they use but do not own.

Association / dependency (uses).

- `CLIController` creates and uses `PromptBuilder`, `FileManager` and `GuiHttpServer`.
- `LLMClient` uses `ResponseParser` and `EnvLoader`.
- `GuiHttpServer` uses `makeUnifiedDiff` and `CodeAgentError`.

14.6.2 Encapsulation and Access Modifiers

Every class exposes the smallest useful public interface. Internal steps are `private`; constructors meant only for subclasses are `protected`. The most important safety rule of the project, *never overwrite a file without asking*, is enforced by encapsulation: the only public write paths (`saveFile`, `writeFile`) always call the private `requestOverwritePermission` before the private `writeAtomically`.

```cpp
class FileManager {
public:
    enum class WriteResult { Written, InvalidPath, Declined, Failed };
    enum class PermissionPrompt { Ask, AlreadyGranted };

    FileManager();
    FileManager(std::istream& input, std::ostream& output, std::ostream& error);

    bool readFile(const std::filesystem::path& filePath, std::string& content) const;
    bool writeFile(const std::filesystem::path& filePath, const std::string& content)
        const;
    WriteResult saveFile(const std::filesystem::path& filePath, const std::string&
        content,
                         PermissionPrompt prompt = PermissionPrompt::Ask) const;
    bool isValidSourceFile(const std::filesystem::path& filePath) const;

private:
    enum class PermissionType { Read, Write };
    bool requestPermission(const std::filesystem::path&, PermissionType) const;
    bool requestOverwritePermission(const std::filesystem::path&) const;
    void writeAtomically(const std::filesystem::path&, const std::string&) const;
    bool isSupportedFileName(const std::filesystem::path&) const;

    std::filesystem::path permissionFilePath_; // .codeagent/permissions.txt
    std::istream& input_;
    std::ostream& output_;
    std::ostream& error_;
};
```

Listing 14.3: Access modifiers in `FileManager` (abridged)

Access level	Example	Purpose
`public`	`LLMClient::generate`, `GuiHttpServer::start`	Minimal interface used by other modules
`protected`	`CodeAgentError( userMessage, diagnostic)`, `NetworkError(msg, diag)`	Only subclasses may construct the abstract base or reuse a constructor
`private`	`FileManager:: writeAtomically`, `RateLimitError:: quotaExhausted_`	Implementation details and invariants
Anonymous `namespace`	`performHttpRequest`, `resolveGuiPort`, `readRegularFile`	File-private helper functions (encapsulation at translation-unit level)
Pimpl (`class Impl;`)	`GuiHttpServer`	Hides all server state and the `httplib.h` dependency from the public header
`const` member functions	`PromptBuilder::build GenerationPrompt(...) const`	Guarantees that queries do not modify the object

Table 14.4: Access control used in the project

14.6.3 Abstraction

Callers work with simple concepts and never see implementation details:

- `LLMClient::generate(prompt, options)` returns text. libcurl, HTTP headers, JSON and finish reasons are hidden.
- `makeUnifiedDiff(a, b)` returns a diff. The LCS algorithm (Hirschberg) is hidden.
- `NativeDialogProvider::choose(kind)` returns a path. Whether zenity or PowerShell ran is hidden.

14.6.4 Polymorphism and Virtual Functions

Abstract base class with pure virtual functions. `CodeAgentError` declares `code()` and `httpStatus()` as pure virtual, so it cannot be instantiated. Each concrete subclass overrides them, and `final` prevents further derivation where that would be meaningless.

```
class CodeAgentError : public std::runtime_error {
public:
    virtual const char* code() const noexcept = 0; // pure virtual
    virtual int httpStatus() const noexcept = 0; // pure virtual
    const std::string& diagnostic() const noexcept;
protected:
    CodeAgentError(const std::string& userMessage, std::string diagnostic);
private:
    std::shared_ptr<const std::string> diagnostic_; // copying cannot throw
};

class FileNotFoundError final : public FileError {
public:
    FileNotFoundError(FileOperation operation, std::filesystem::path path,
                      std::error_code error);
    const char* code() const noexcept override; // "FILE_NOT_FOUND"
    int httpStatus() const noexcept override; // 404
};
```

Listing 14.4: Abstract base class and an overriding subclass

Dynamic dispatch in practice. The HTTP server converts *any* error into a JSON response with one function and no type checks. Adding a new error class requires no change here (the open/closed principle):

```
void respondWithError(httplib::Response& response, const CodeAgentError& error) {
    logError(std::cerr, error); // redacted diagnostic -> terminal
    setJsonError(response, error.httpStatus(), // virtual: 404, 429, 504, ...
                 error.code(), error.what()); // virtual code, safe message
}
```

Listing 14.5: One handler for every error type through virtual calls

Catching by base-class reference is polymorphic too. `catch (const NetworkError&)` also catches `TimeoutError`, and `CLIController::run` catches every project error with one `catch (const CodeAgentError&)`.

Interface polymorphism.

```cpp
class NativeDialogProvider {
public:
    virtual ~NativeDialogProvider() = default; // safe deletion via base pointer
    virtual NativeDialogResult choose(NativeDialogKind kind) = 0;
};
// Production (src/NativeDialog.cpp): runs zenity / PowerShell without a shell
class SystemNativeDialogProvider final : public NativeDialogProvider {
public:
    NativeDialogResult choose(NativeDialogKind kind) override;
};
// Tests (tests/GuiHttpServerTest.cpp): returns scripted results
class ScriptedNativeDialogProvider final : public NativeDialogProvider { /* ... */ };
```

Listing 14.6: Interface with a virtual destructor and two implementations

Compile-time polymorphism.

- Function overloading: four `LLMClient::generateContent` and `generateCompleteContent` overloads, and five `GuiHttpServer` constructors.
- Template instantiation: see Section 14.8.

14.6.5 Dependency Injection and Design Patterns

Classes receive their collaborators instead of creating them. This makes every component replaceable and lets all tests run offline (requirement R8).

Class	Injected dependency	Production	Test
`LLMClient`	`HttpTransport` (`std::function`)	libcurl `performHttpRequest`	lambda returning canned JSON
`CLIController`	`LLMCall`, streams, `GuiCommand`	`LLMClient`, `cin`/`cout`/`cerr`	fake LLM, string streams
`GuiHttpServer`	handlers, `NativeDialogProvider`, clock	lambdas calling the core, zenity provider, `steady_clock`	fakes, scripted provider, fake clock
`FileManager`	streams	terminal	string streams

Table 14.5: Dependency injection points

Design patterns used:

Pimpl `GuiHttpServer::Impl`.

Strategy Injected `std::function` objects and the dialog provider.

Factory `createNativeDialogProvider()` and the named constructor `ConfigurationError::missingSetting()`.

RAII / scope guard See Section 14.8.

Facade `LLMClient::generate`.

State machine Proposal status, on the server and in `proposalWorkflow.js`.

Composition root `runGuiCommand` wires the real implementations together.

LRU cache `std::list` plus a `std::map` of iterators for the documents opened in a session.

14.7 UML Diagrams

The repository file `UML-diagrams.md` contains the complete Mermaid sources, kept consistent with the code. The class diagram is split into two views (application classes, Figure 14.1, and the error hierarchy, Figure 14.2), followed by three sequence diagrams and the proposal state diagram. In the class diagrams, `+` marks public and `-` private members, `*` marks a pure virtual function, and the stereotypes `<<abstract>>` and `<<interface>>` mark classes that cannot be instantiated.

From	Relationship	To	Meaning
CodeAgentError subclasses	Generalisation	CodeAgentError	is-a (inheritance)
SystemNative DialogProvider	Realisation	NativeDialogProvider	implements the interface
GuiHttpServer	Composition	Impl	owns (`unique_ptr`)
GuiHttpServer::Impl	Aggregation	NativeDialogProvider	shares (`shared_ptr`)
CLIController	Dependency	PromptBuilder, FileManager, LLMClient, GuiHttpServer	creates and uses
LLMClient	Dependency	ResponseParser, EnvLoader	uses
GuiHttpServer	Dependency	makeUnifiedDiff, CodeAgentError	uses

Table 14.6: UML relationships between classes

14.8 Advanced C++ Mechanics

14.8.1 Exception Handling (try / throw / catch)

Errors are *thrown* as typed exceptions at the point of failure and *caught* at well-defined boundaries:

- `main` (start-up failures);
- `CLIController::run` (any command failure becomes a message and exit code 1);
- each HTTP route handler and the server-wide exception handler (any failure becomes a JSON error and the server keeps running).

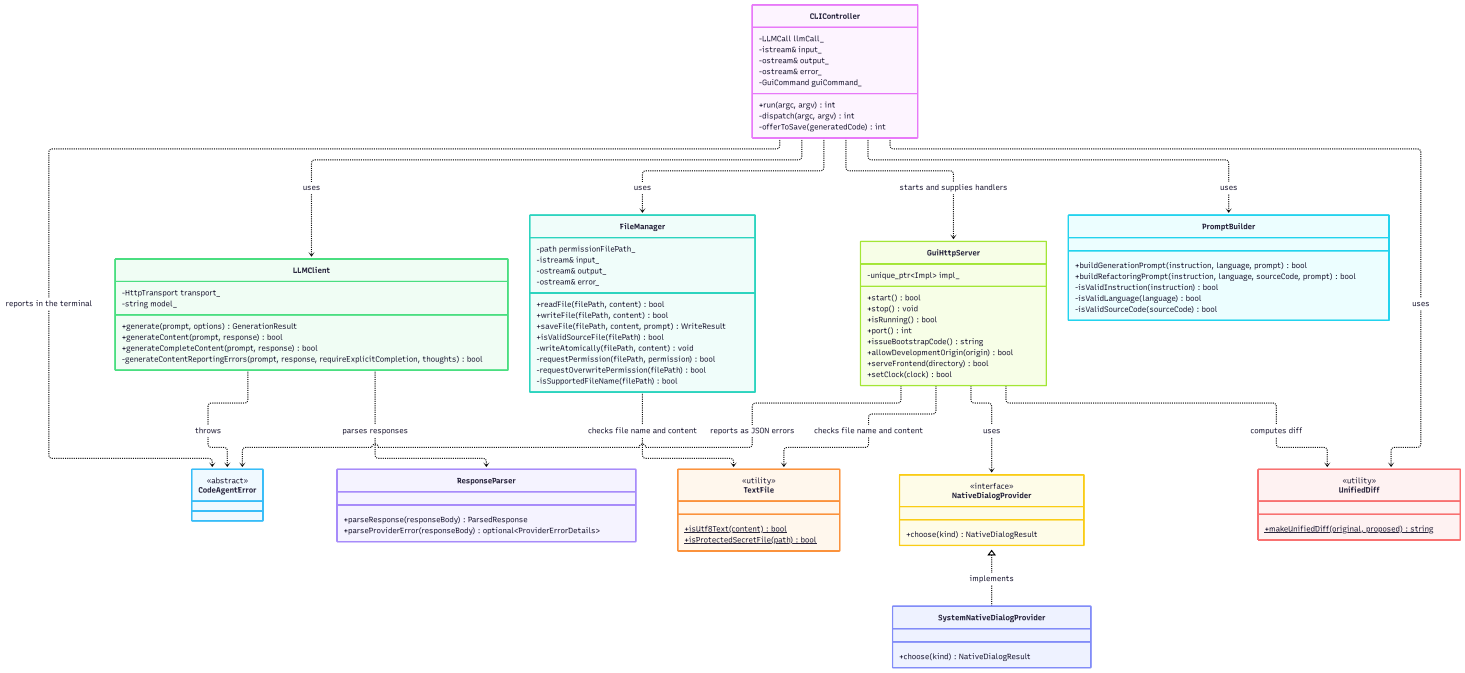


Figure 14.1: UML class diagram, view 1: application classes and their dependencies

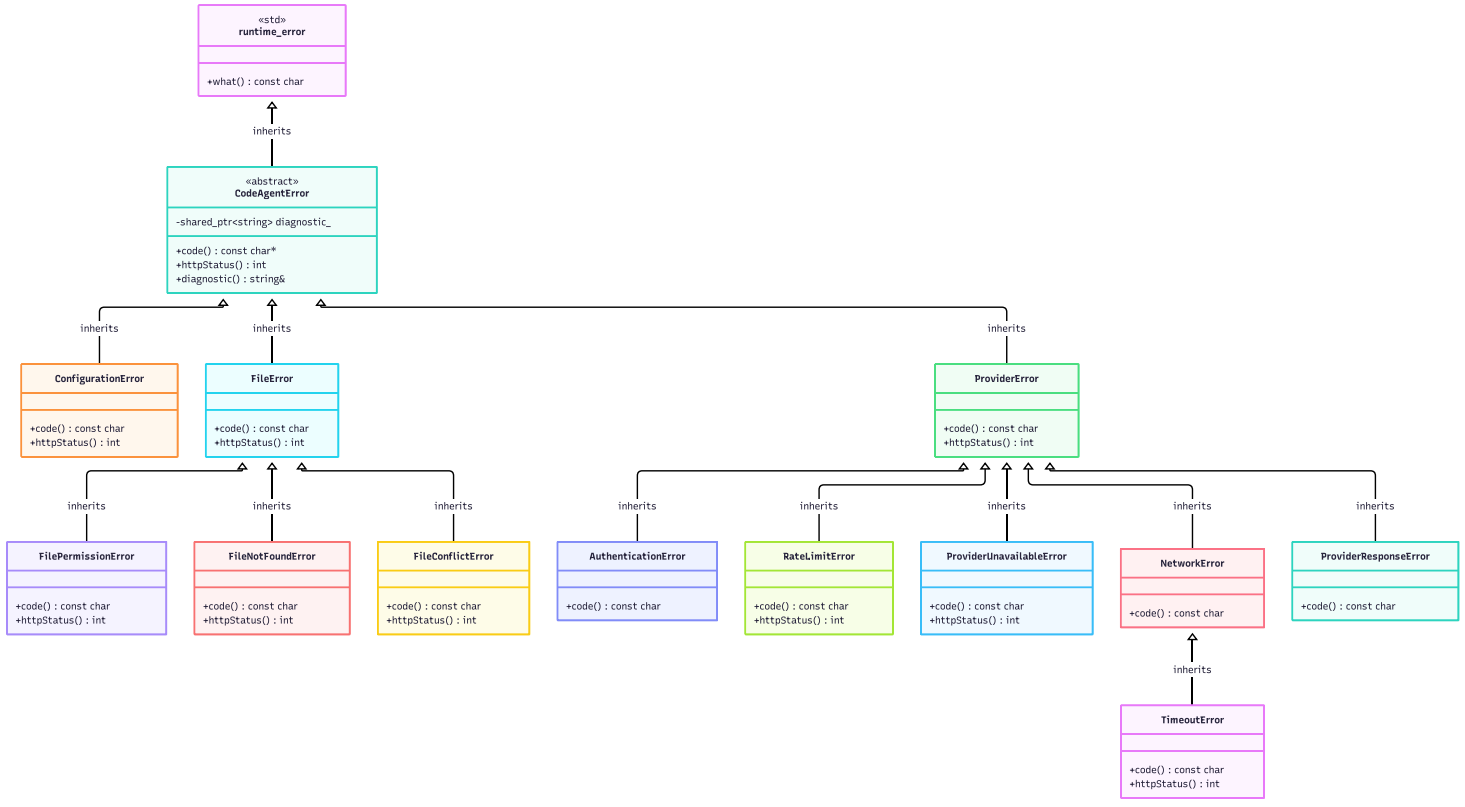


Figure 14.2: UML class diagram, view 2: the `CodeAgentError` exception hierarchy

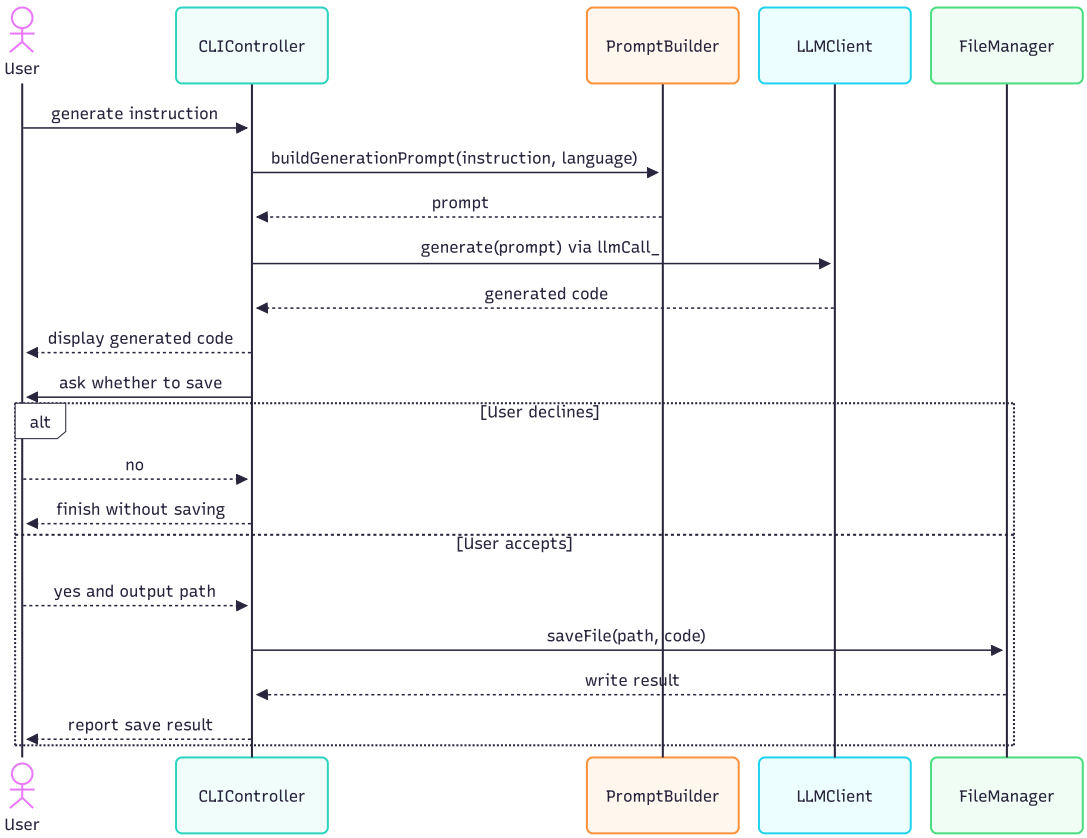


Figure 14.3: UML sequence diagram: CLI code generation with optional save

```cpp
[[noreturn]] void throwHttpError(long httpStatus, const std::string& responseBody,
                                 const std::string& model) {
    const auto details = ResponseParser().parseProviderError(responseBody);
    const std::string diagnostic = describeProviderError(httpStatus, details);
    const std::string status = details ? details->status : "";
    const std::string reason = details ? details->reason : "";
    if (httpStatus == 401 || httpStatus == 403 || status == "UNAUTHENTICATED" ||
        status == "PERMISSION_DENIED" || reason == "API_KEY_INVALID") {
        throw AuthenticationError(diagnostic);
    }
    if (httpStatus == 429 || status == "RESOURCE_EXHAUSTED") {
        const bool quotaExhausted = details && hasDailyQuotaViolation(*details);
        throw RateLimitError(quotaExhausted, details ? details->retryDelay : std::
            nullopt,
                             diagnostic);
    }
    if (httpStatus == 408 || httpStatus == 504 || status == "DEADLINE_EXCEEDED") {
        throw TimeoutError(diagnostic);
    }
    if (httpStatus >= 500) throw ProviderUnavailableError(diagnostic);
    // ... 404 -> ProviderError("model not found"), others -> ProviderError
}
```

Listing 14.7: Throwing typed exceptions from HTTP status codes (`LLMClient.cpp`, abridged)

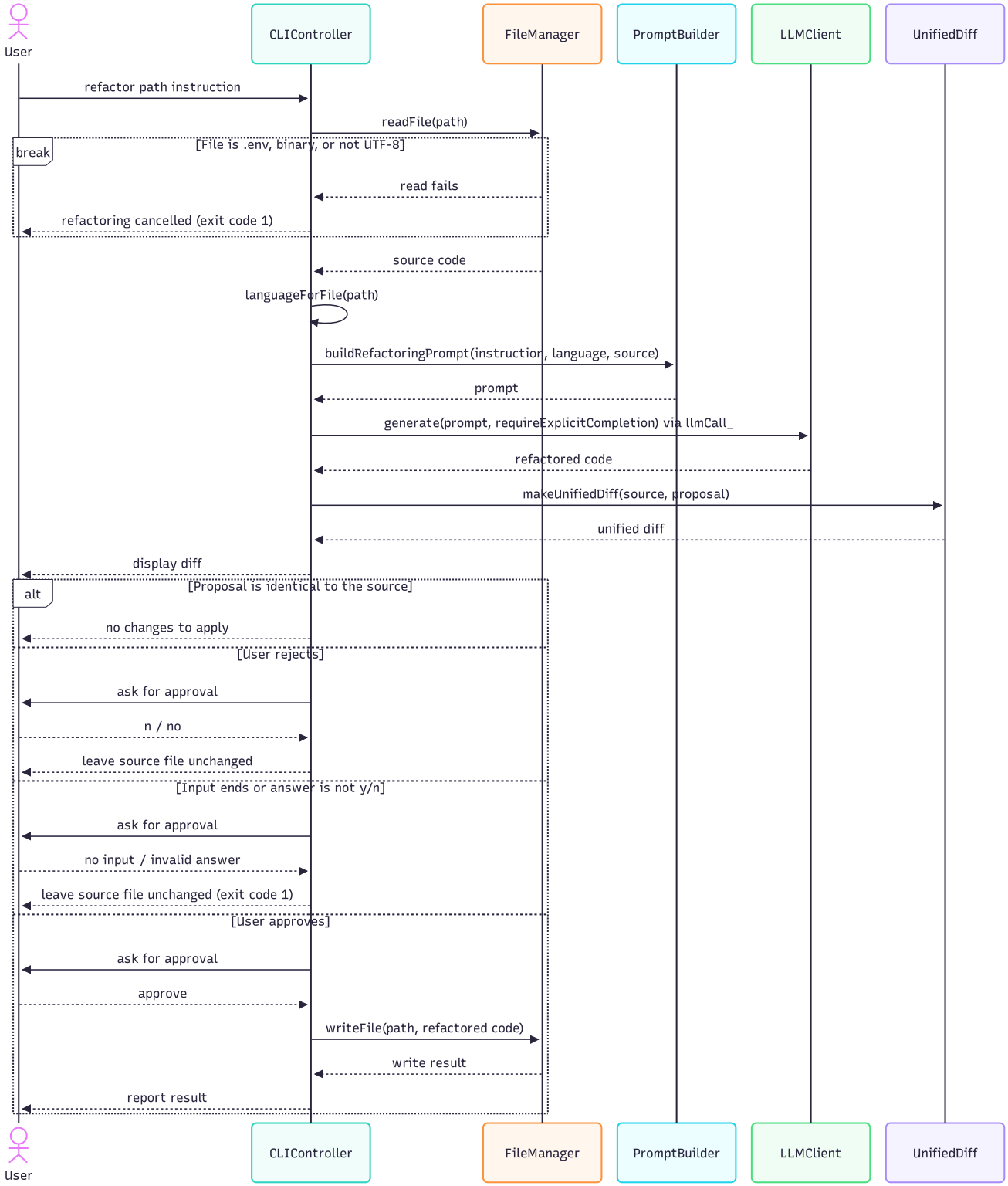


Figure 14.4: UML sequence diagram: CLI refactoring with diff and approval

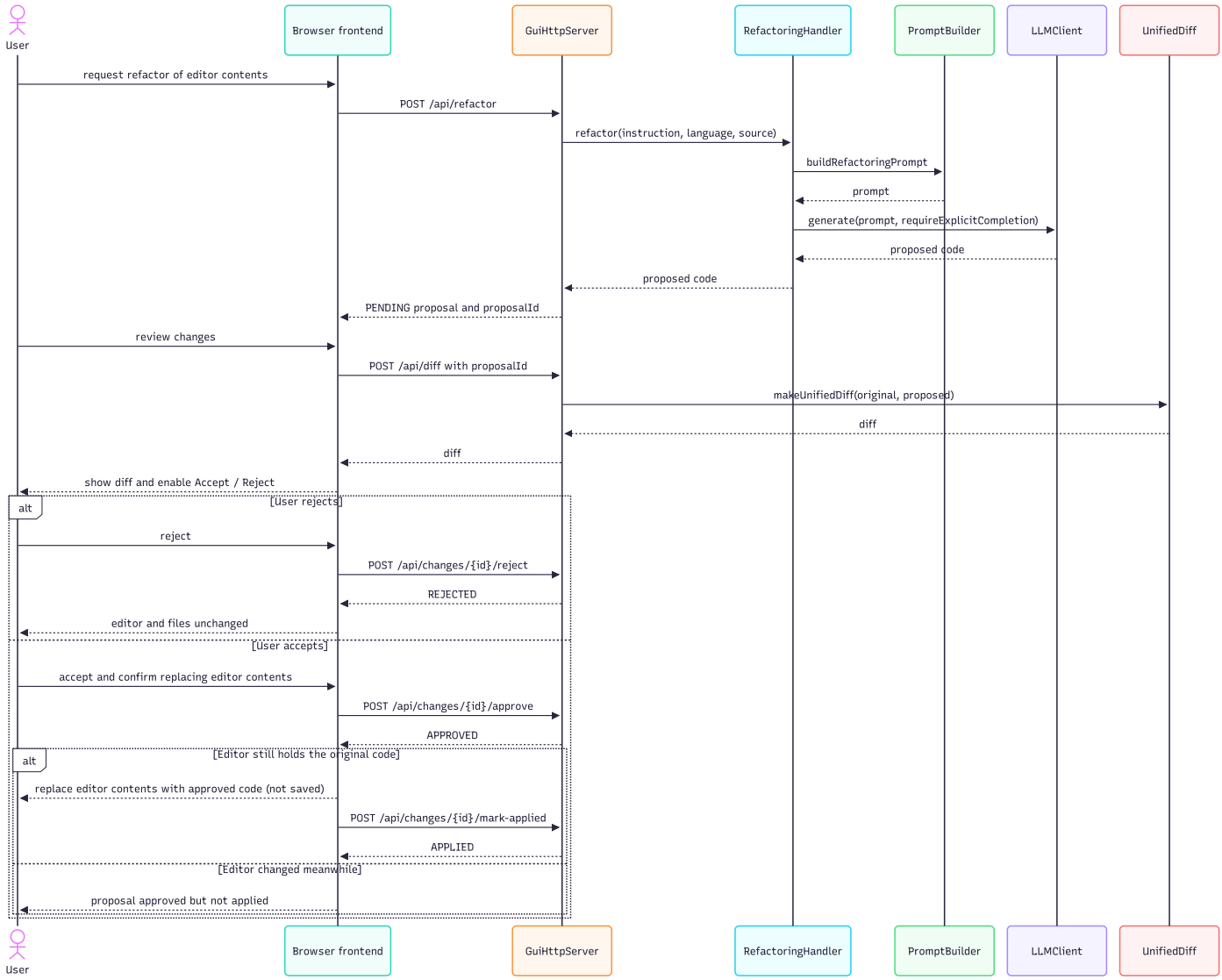


Figure 14.5: UML sequence diagram: GUI refactoring proposal lifecycle

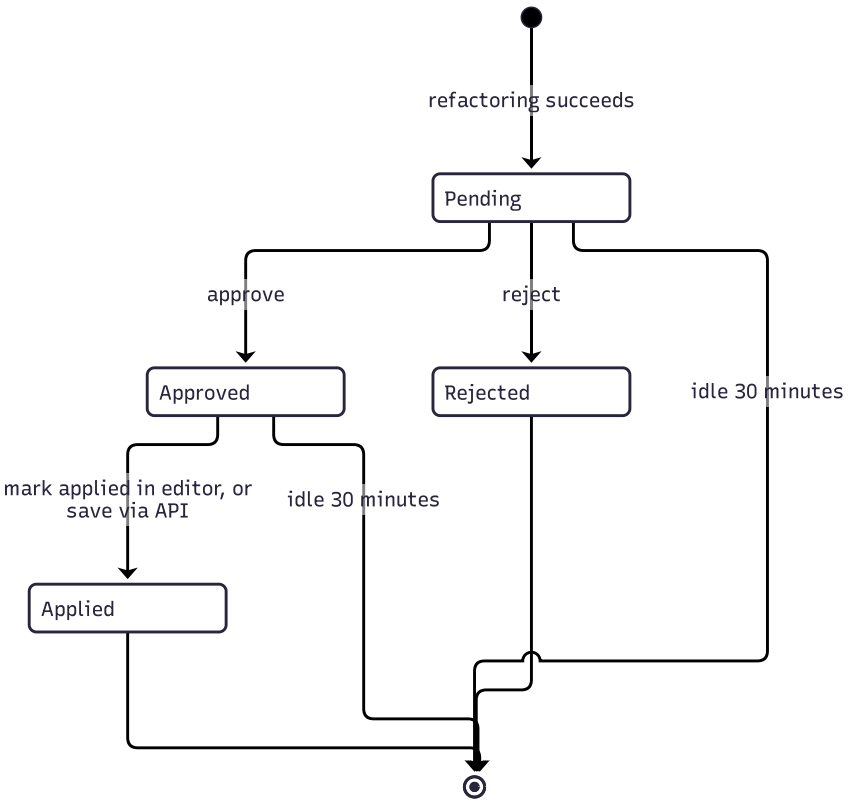


Figure 14.6: UML state diagram: refactoring proposal states

```cpp
int CLIController::run(int argc, char* argv[]) {
    try {
        return dispatch(argc, argv);
    } catch (const CodeAgentError& failure) { // all project errors (polymorphic)
        error_ << failure.what() << '\n';
        if (!failure.diagnostic().empty())
            error_ << "Details: " << failure.diagnostic() << '\n';
    } catch (const std::bad_alloc&) { // out of memory
        error_ << "Error: CodeAgent ran out of memory. ...\n";
    } catch (const std::exception& failure) { // anything else, secrets redacted
        error_ << "Error: An unexpected internal error occurred ("
               << redactSecrets(failure.what()) << "). Please retry.\n";
    }
    return 1;
}
```

Listing 14.8: The CLI error boundary: catch from most specific to most general

Advanced exception techniques used:

- `[[noreturn]]` functions such as `throwHttpError` and `throwFileSystemError`, which maps `std::error_code` values to the right subclass.
- `noexcept` on getters and logging functions.
- `std::exception_ptr` with `std::current_exception` and `std::rethrow_exception` in `logUnexpectedException` and the server's exception handler.

- **Exception-safe copies**: exception members are stored as `std::shared_ptr<const T>`, because copying an exception during throw must not itself throw.
- **Secret redaction at construction**: every diagnostic passes through `redactSecrets`, so no catch site can leak the API key.

14.8.2 Memory and Resource Management (RAII and Smart Pointers)

The code base contains **no raw `new` or `delete`**. Every resource is owned by an object whose destructor releases it. C++ guarantees that destructors run on every exit path, including exceptions.

RAII owner	Resource	Location
`std::unique_ptr<CURL, CurlEasyDeleter>`	libcurl request handle	`LLMClient.cpp`
`std::unique_ptr<curl_slist, CurlHeaderListDeleter>`	libcurl header list	`LLMClient.cpp`
`TemporaryFileGuard`	temporary file on disk	`FileManager.cpp`
`FileDescriptor`	POSIX file descriptor	`GuiHttpServer.cpp`
`ProposalCapacity Reservation`	reserved proposal slot	`GuiHttpServer.cpp`
`std::lock_guard<std:: mutex>`	mutex lock	`GuiHttpServer.cpp`
`std::unique_ptr<Impl>` + destructor calling `stop()`	server socket and listener thread	`GuiHttpServer.cpp`
`std::shared_ptr< NativeDialogProvider>`	shared dialog provider	`GuiHttpServer.hpp`

Table 14.7: RAII and smart-pointer ownership

```
class TemporaryFileGuard {
public:
    explicit TemporaryFileGuard(std::filesystem::path path) : path_(std::move(path))
        {}
    ~TemporaryFileGuard() {
        if (!released_) {
            std::error_code ignored;
            std::filesystem::remove(path_, ignored); // never throws in a destructor
        }
    }
    TemporaryFileGuard(const TemporaryFileGuard&) = delete; // non-copyable
    TemporaryFileGuard& operator=(const TemporaryFileGuard&) = delete;
    void release() noexcept { released_ = true; } // called after a successful rename
private:
    std::filesystem::path path_;
    bool released_ = false;
};
```

Listing 14.9: A scope guard: the temporary file is removed on any failure path

Further memory-related techniques:

- **Move semantics** (`std::move`) transfer large strings, such as generated code and response bodies, without copying.
- C++17 **map node handles** (`extract`/`insert`) swap proposals without allocation, and restore the old entry if insertion fails.
- **Bounded buffers**: a 2 MiB limit on Gemini responses, a 1 MiB limit on HTTP requests, 512 KiB editor files, 128 proposals and 256 remembered documents.

14.8.3 Templates and Generic Programming

The project relies heavily on C++ templates through the standard library and its dependencies, choosing template arguments to model ownership and policy:

- **Class templates with a policy parameter:** `std::unique_ptr<CURL, CurlEasyDeleter>` combines the generic smart pointer with a custom deleter type, so one template manages a C resource.
- **Type-erasing class template:** `std::function<bool(const std::string&, ...)>` stores any callable (function, lambda, functor). It is used for `HttpTransport`, `LLMCall`, `GenerationHandler`, `RefactoringHandler` and `Clock`.
- **Vocabulary templates:** `std::optional<std::string>` (an optional finish reason), `std::optional<ProviderErrorDetails>`, `std::shared_ptr<const T>`, and `std::atomic<int>`.
- **Container templates:** `std::map`, `std::list`, `std::vector`, `std::unordered_set`, `std::pair` and `std::tuple`. Tuples give lexicographic ordering for proposal eviction.
- **Function templates:** `std::make_unique<Impl>`, `std::make_shared<const std::string>`, and nlohmann/json's `body["x"].get<std::string>()`. Generic algorithms (`std::transform`, `std::all_of`, `std::remove_if`) are parameterised with lambdas.

14.8.4 Operator Overloading

- **Function-call operator** `operator()` is overloaded to create function objects (functors) that act as custom deleters:

```cpp
struct CurlEasyDeleter {
    void operator()(CURL* handle) const noexcept { curl_easy_cleanup(handle); }
};
struct CurlHeaderListDeleter {
    void operator()(curl_slist* list) const noexcept { curl_slist_free_all(list
        ); }
};
using CurlHandle = std::unique_ptr<CURL, CurlEasyDeleter>;
using CurlHeaderList = std::unique_ptr<curl_slist, CurlHeaderListDeleter>;
```

Listing 14.10: Overloaded `operator()` used as a smart-pointer deleter

- **Deleted assignment operators** (`operator=(const T&) = delete`) on `GuiHttpServer`, `TemporaryFileGuard`, `FileDescriptor` and `ProposalCapacityReservation`. They prevent two objects from owning one socket, file or slot.
- Use of library-overloaded operators: stream insertion `operator<<` for all terminal output, `std::string operator+`, `nlohmann::json::operator[]`, and the `operator bool` of `std::function` and `std::optional`.

14.8.5 Concurrency

- The GUI server answers requests on a worker-thread pool. Shared state is protected by separate mutexes: `configMutex_`, `proposalMutex_`, `saveMutex_`, `documentsMutex_`, `sessionMutex_` and `lifecycleMutex_`.
- `std::call_once` initialises libcurl exactly once, and a function-local `static` loads the `.env` files once, thread-safely.
- Ctrl+C is handled with a `volatile std::sig_atomic_t` flag, the only async-signal-safe way to communicate with the main loop.
- Tests prove correctness under contention: `ConcurrentApprovalAndRejectionHaveOneWinner` and `ConcurrentRefactorsCannotOverbookLastCapacitySlot`.

14.8.6 Other Modern C++ Features

C++17 `std::filesystem` (`canonical`, `symlink_status`, `lexically_normal`), `std::chrono::steady_clock` timeouts, `enum class` for every closed set of outcomes, `constexpr` constants, delegating and `explicit` constructors, `override` and `final`, raw string literals, if-with-initialiser, lambdas with explicit captures, and type aliases (`using`).

14.9 Frontend and Backend Integration

14.9.1 Separation of Concerns

- **Backend (C++)** holds all business logic: prompts, Gemini calls, response validation, diffs, proposal state, file I/O and security.

- **CLI frontend (C++)**: `CLIController` handles only argument parsing and terminal interaction.
- **GUI frontend (JavaScript)** holds only user-interface state:
  - `api.js` is the single module that performs HTTP requests and validates every response;
  - `proposalWorkflow.js` is the client-side review state machine;
  - `diffView.js` renders diffs safely with `textContent`;
  - `app.js` wires the user interface together.
- **Shared core:** both front ends use the same `PromptBuilder`, `LLMClient` and `ResponseParser`. The GUI reaches them through handler lambdas defined in `CLIController.cpp`, so the Gemini integration is never duplicated.

14.9.2 Communication Protocol and Security

The browser and the C++ server communicate with JSON over HTTP on `127.0.0.1`. Start-up uses a one-time handshake:

1. `codeagent gui` opens `http://127.0.0.1:8080/#bootstrap=<code>`. The URL fragment is never sent to servers.
2. The page sends `POST /api/session {"bootstrap": code}`. The server checks the single-use code (valid for 5 minutes, compared in constant time) and returns a 256-bit session token, plus an `HttpOnly; SameSite=Strict` cookie for page reloads.
3. Every later request carries `X-CodeAgent-Token`. Before any route runs, a pre-routing handler verifies the Host header (against DNS rebinding), the Origin header (against other websites) and the token.

```
async function requestJson(path, { method = 'GET', body, session = false } = {}) {
  const options = { method, headers: { Accept: 'application/json' } };
  if (session) options.credentials = 'include';
  else if (sessionToken !== null) options.headers['X-CodeAgent-Token'] = sessionToken
      ;
  if (body !== undefined) {
    options.headers['Content-Type'] = 'application/json';
    options.body = JSON.stringify(body);
  }
  const response = await fetchImpl(`${normalizedBaseUrl}${path}`, options);
  const payload = JSON.parse(await response.text());
  if (!response.ok) {
    const code = payload?.error?.code ?? 'HTTP_ERROR';
    if (code === 'UNAUTHORIZED') sessionToken = null;
    throw new ApiError(code, payload?.error?.message, response.status);
  }
  return payload;
}
```

Listing 14.11: The single HTTP function of the frontend (`frontend/js/api.js`, abridged)

14.9.3 HTTP API

Endpoint	Purpose	Side effect
`POST /api/session`	exchange bootstrap code or cookie for the token	consumes the code
`GET /api/health`, `GET /api/config`	status and directories	none
`POST /api/generate`	generate code	calls Gemini
`POST /api/refactor`	create a refactoring proposal	calls Gemini; proposal `PENDING`
`POST /api/diff`	unified diff of a proposal or two buffers	none
`POST /api/changes/{id}/approve`	approve a proposal (does *not* save)	`PENDING` → `APPROVED`
`POST /api/changes/{id}/reject`	reject a proposal	`PENDING` → `REJECTED`
`POST /api/changes/{id}/mark-applied`	proposal applied in the editor	`APPROVED` → `APPLIED`
`POST /api/files/open-dialog`	native file picker	starts a child process
`POST /api/files/read`	open a file in the editor	reads a file
`POST /api/files/write`	save an opened file in place (conflict detection)	writes a file atomically
`POST /api/output/select-directory`	choose the output directory	session state
`POST /api/save`	Save As / save an approved proposal	writes a new file

Table 14.8: HTTP API implemented by `GuiHttpServer`

Every route checks the content type, validates the *exact* JSON shape (unknown fields such as `apiKey` are rejected), and reports errors in a single format: `{"error":{"code":...,"message":...}}`.

14.9.4 Persistent Storage and File I/O

- **Source files (read):**
  - binary-mode reads that keep CRLF line endings;
  - strict UTF-8 validation;
  - in the GUI, a 512 KiB limit enforced while reading;

  - FIFOs and devices are refused without blocking (`O_NONBLOCK` + `fstat`).

- **Source files (write):**
  - atomic replacement through a temporary file and `rename`;
  - the GUI writer also uses `O_EXCL` temporary files, `fsync`, preservation of permission bits, and `link()` for new files, so a file created concurrently is never overwritten;
  - symbolic-link targets are refused.

- **Change detection:** an FNV-1a 64-bit revision hash returned on open. Saving over a file modified externally (for example in VS Code) requires explicit confirmation (`409 FILE_CHANGED_ON_DISK`).

- **Persistent permission store:** answering *[A] Always allow* at a CLI permission prompt appends `read|path` or `write|path` to `.codeagent/permissions.txt`. Later runs read this file and skip the question. Overwriting still always asks.

- **Configuration storage:** `.env` files are read in priority order: `$CODEAGENT_ENV_FILE`, then `./.env`, then `~/.config/codeagent/.env`. Variables already in the environment are never overwritten.

14.9.5 Gemini Integration

1. `LLMClient::generate` reads `GEMINI_API_KEY`. If it is missing, a `ConfigurationError` is thrown before any network access.
2. It builds the JSON body `{"contents":[{"parts":[{"text": prompt}]}]}`.
3. It POSTs to `https://generativelanguage.googleapis.com/v1beta/models/gemini-3.5-flash:generateContent` through libcurl:
   - the key is sent in the `x-goog-api-key` header, never in the URL;
   - TLS certificates are verified;
   - timeouts are 10 s to connect and 60 s in total.
4. `ResponseParser` validates every level of the reply. Unknown finish reasons are treated as failures (fail-closed). A refactoring must end with `finishReason = STOP`, so truncated code is never offered for saving.

14.10 Testing and Quality Assurance

All tests run offline, using the dependency-injection seams described in Section 14.6. GoogleTest cases are registered individually with CTest through `gtest_discover_tests`.

The full suite passed in our verification run: 214/214 C++ tests (run with `ctest -j1`) and 79/79 frontend tests. The FileManager tests share one sandbox directory, so the suite is currently run serially. Giving each test its own directory is listed as future work.

Test file	Cases	What is verified
`FileManagerTest.cpp`	15	any extension, `.env` refusal, CRLF round-trip, overwrite confirmation, permission errors
`PromptBuilderTest.cpp`	9	templates, empty-input rejection, determinism
`LLMClientTest.cpp`	26	HTTP errors, timeouts, quota, malformed JSON, 2 MiB limit, key never in errors
`ResponseParserTest.cpp`	32	JSON structure, finish reasons, thoughts, fence stripping
`UnifiedDiffTest.cpp`	7	diff correctness, line endings
`EnvLoaderTest.cpp`	8	`.env` parsing and priority
`CLIControllerTest.cpp`	26	approval-gated writes, rejection, invalid input, save prompts
`CodeAgentErrorTest.cpp`	7	hierarchy, codes, redaction
`GuiHttpServerTest.cpp`	84	real HTTP: authentication, Host/Origin, every endpoint, proposal states, concurrency, conflicts, permissions
C++ total	**214**	`ctest --test-dir build --output-on-failure`
`frontend/tests/*.test.mjs`	79	API validation, workflow gating, diff rendering, real `app.js` on a test DOM

Table 14.9: Automated test suites

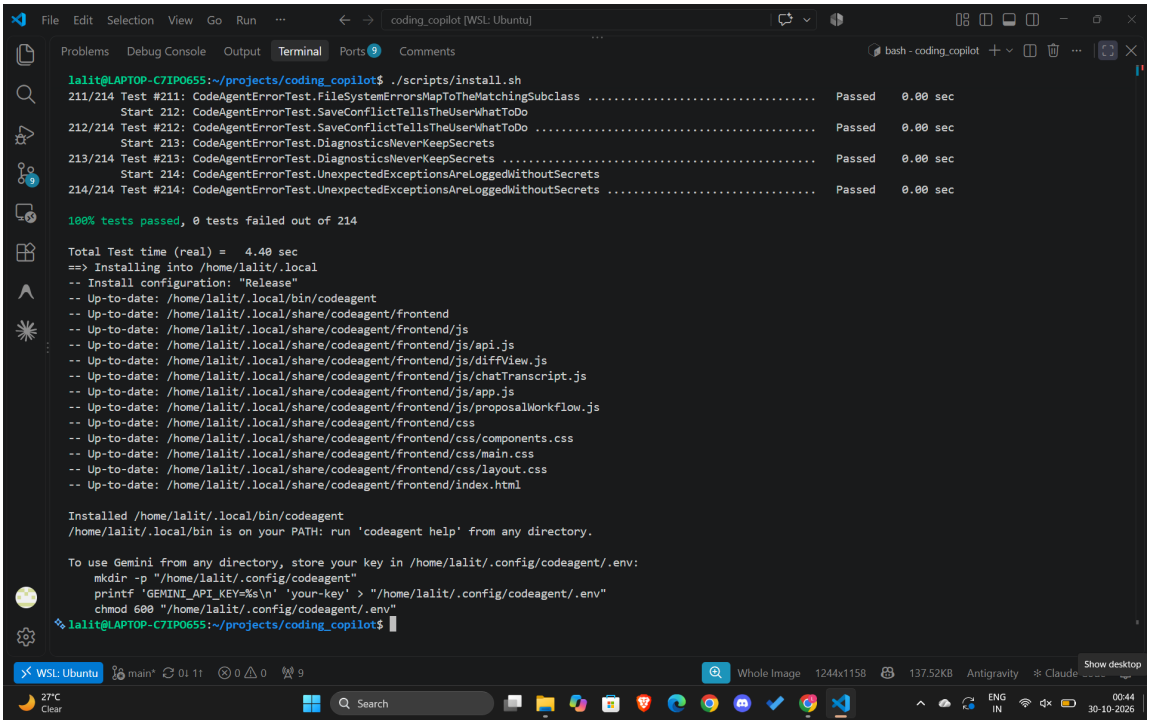


Figure 14.7: C++ test results: `scripts/install.sh` runs all 214 CTest tests (100% passed) before installing `codeagent`

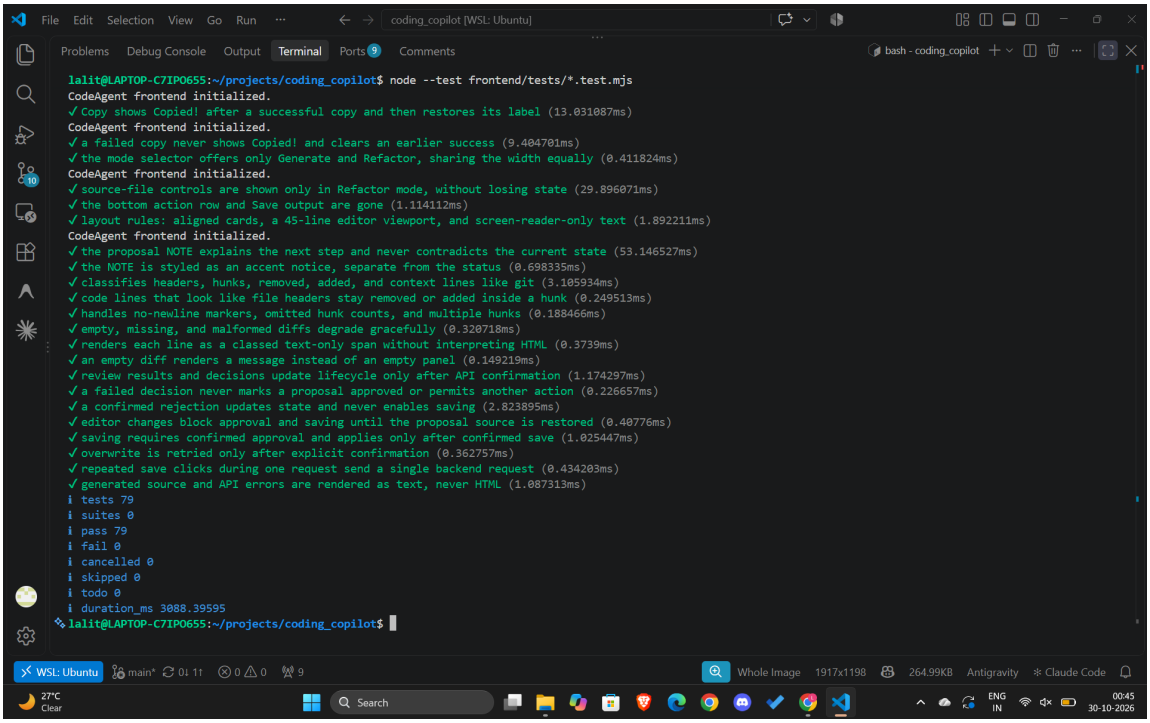


Figure 14.8: Frontend test results (Node.js)

14.11 Build System

CMake 3.15+ builds the project with C++17 (`CMAKE_CXX_STANDARD 17`, extensions off) and warnings (`-Wall -Wextra -Wpedantic`).

- libcurl and nlohmann/json come from the system.
- GoogleTest v1.14.0 and cpp-httplib v0.58.0 are fetched at pinned versions with `FetchContent`.
- Static libraries: `CodeAgentError`, `UnifiedDiff`, `GuiHttpServer`.
- The `codeagent` executable links them.
- `install()` rules and `scripts/install.sh` install the binary and frontend to `~/.local`, so `codeagent` works from any folder.

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
node --test frontend/tests/*.test.mjs
export GEMINI_API_KEY=... # or put it in .env
./build/codeagent help
./build/codeagent generate "a function that checks whether a string is a palindrome"
./build/codeagent refactor src/example.cpp "improve error handling"
./build/codeagent gui
```

Listing 14.12: Build, test and run

14.12 Version Control and Collaboration

The project was developed in Git with feature branches, integration branches and frequent, small commits that follow the Conventional Commits style (`feat(gui):`, `fix:`, `docs:`, `test:`, `chore:`).

Metric	**Value**
Total commits	61 (Lalitadithya Kariyam: 30, Samuel John: 31)
Lines added / deleted (non-merge)	Lalitadithya: +9,644 / −966; Samuel: +13,385 / −1,721
Branches	`main`, `feature/gui-backend`, `feature/gui-frontend`, `feature/setup-googletest`, `integration/gui-complete`, `integration/gui-full-app`
Merge commits	`225ba46` (frontend into backend), `cb6e3f6`, `bfa1bad` (integration into `main`)
Development period	1 October 2026 – 29 October 2026

Table 14.10: Repository statistics

Parallel collaboration. On 26–28 October both members committed on the same days (25 commits on 27 October alone). The backend and frontend branches advanced in parallel, and each backend endpoint was followed by its frontend integration. Examples: `c30c7cf` *add GUI code generation endpoint* → `7700d60` *connect code generation to backend*; and `42b433b` *add GUI refactoring endpoint* → `3d038e1` *integrate proposal review workflow*. The frontend branch was merged into the backend branch (`225ba46`) so the UI was built against the real API, then integrated into `main` (`bfa1bad`).

Phase	**Lalitadithya Kariyam (251130)**	**Samuel John (251118)**
Foundation (1–13 Oct)	CLI foundation & command controller (`a36e672`), `planning.md`	`FileManager` (`61d8e20`), GoogleTest setup (`6666888`)
Core AI (19–24 Oct)	`PromptBuilder` (`74e1533`), Gemini integration (`137a6be`)	`ResponseParser` (`35d584d`), file-safety fixes (`5fea4c6`, `a72d40d`)
GUI backend (26–27 Oct)	API contract (`47789e6`), HTTP server (`cc7f573`), health/config (`566250f`), `gui` command (`31ee0da`), generation endpoint (`c30c7cf`), safe saving (`8fd772a`)	CLI diff & approval (`43e5c87`), refactoring endpoint (`42b433b`), diff/approval workflow (`700aebe`), native dialogs (`4616f81`)
GUI frontend (26–28 Oct)	API client (`270ec51`), backend status (`3b21ab0`), secure integration (`6df7b43`), chat with thoughts (`1f45db5`), Save As UX (`a319893`)	HTML structure & CSS (`b3b1afe`, `42eca48`), editor workspace (`9527284`), proposal review UI (`95ccde4`, `3d038e1`), frontend serving (`0eac287`)
Hardening (28–29 Oct)	Integration merge (`bfa1bad`), UML diagrams, README	Exception hierarchy (`a077a64`), interactive save (`dc4a225`), global install (`62cdde7`), any-extension files (`522b973`)

Table 14.11: Contribution timeline (selected commits)

14.13 Results

14.13.1 Command-Line Interface

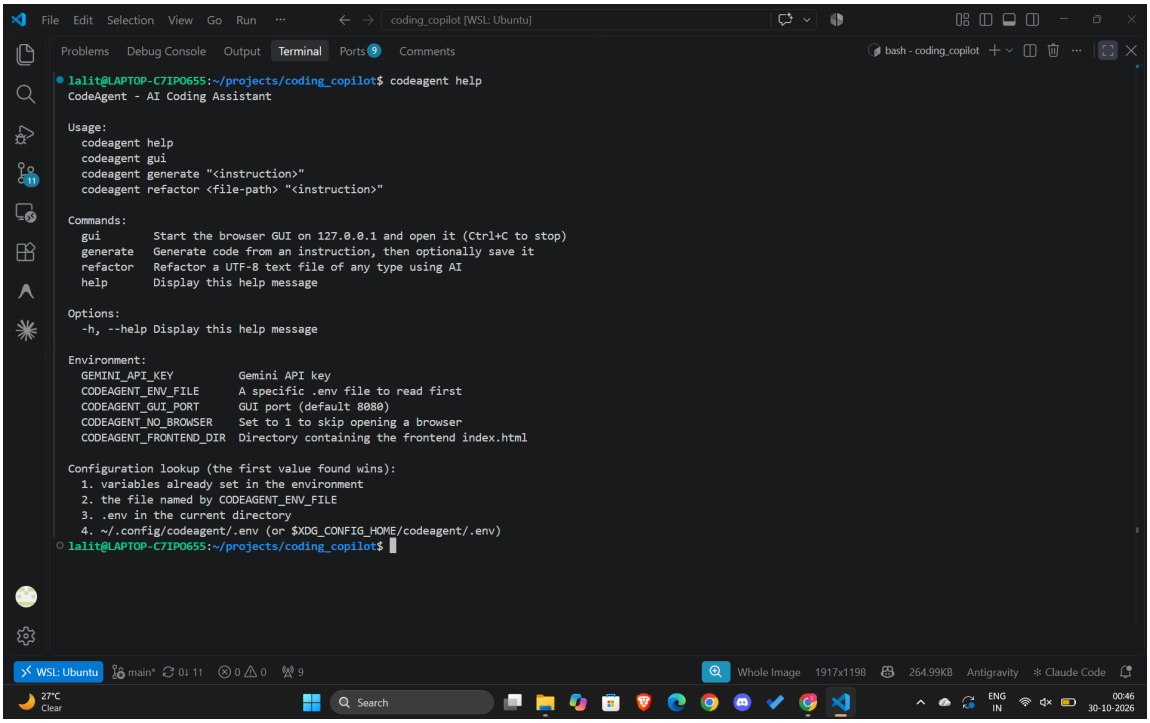


Figure 14.9: CLI help output (`codeagent help`)

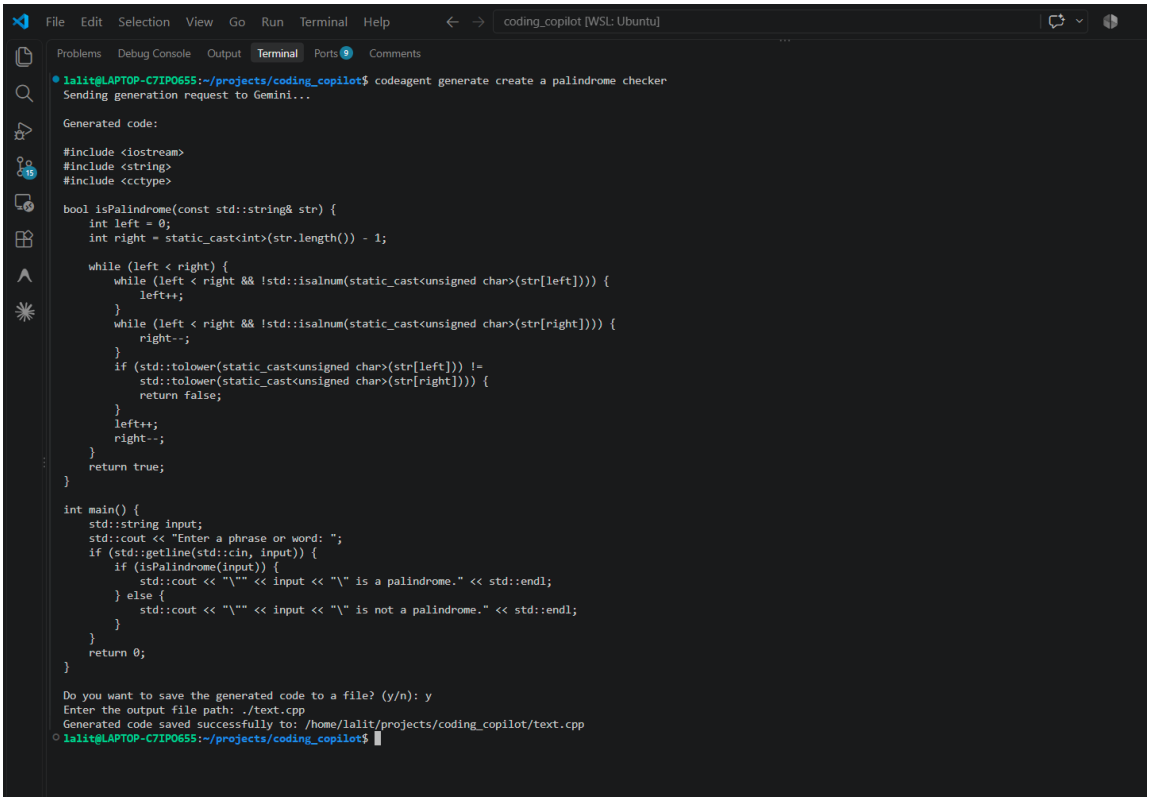


Figure 14.10: CLI code generation: generated code is displayed, then saved to a user-chosen path

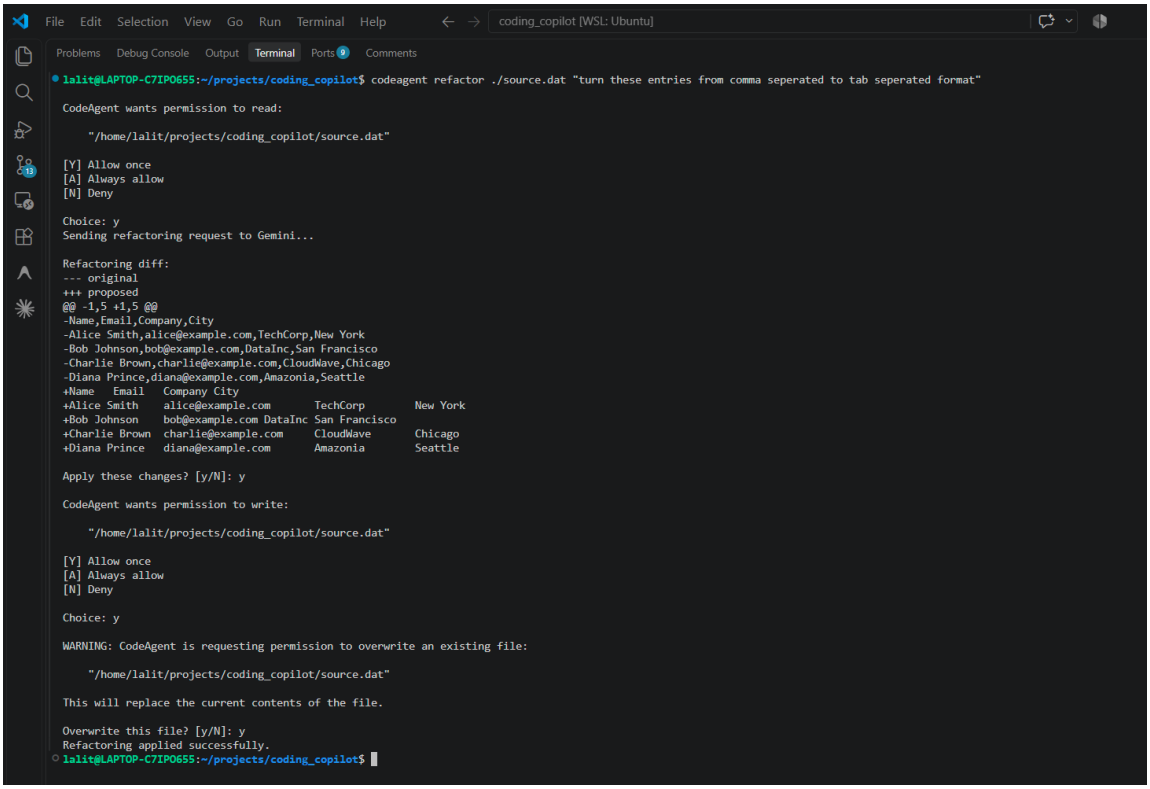
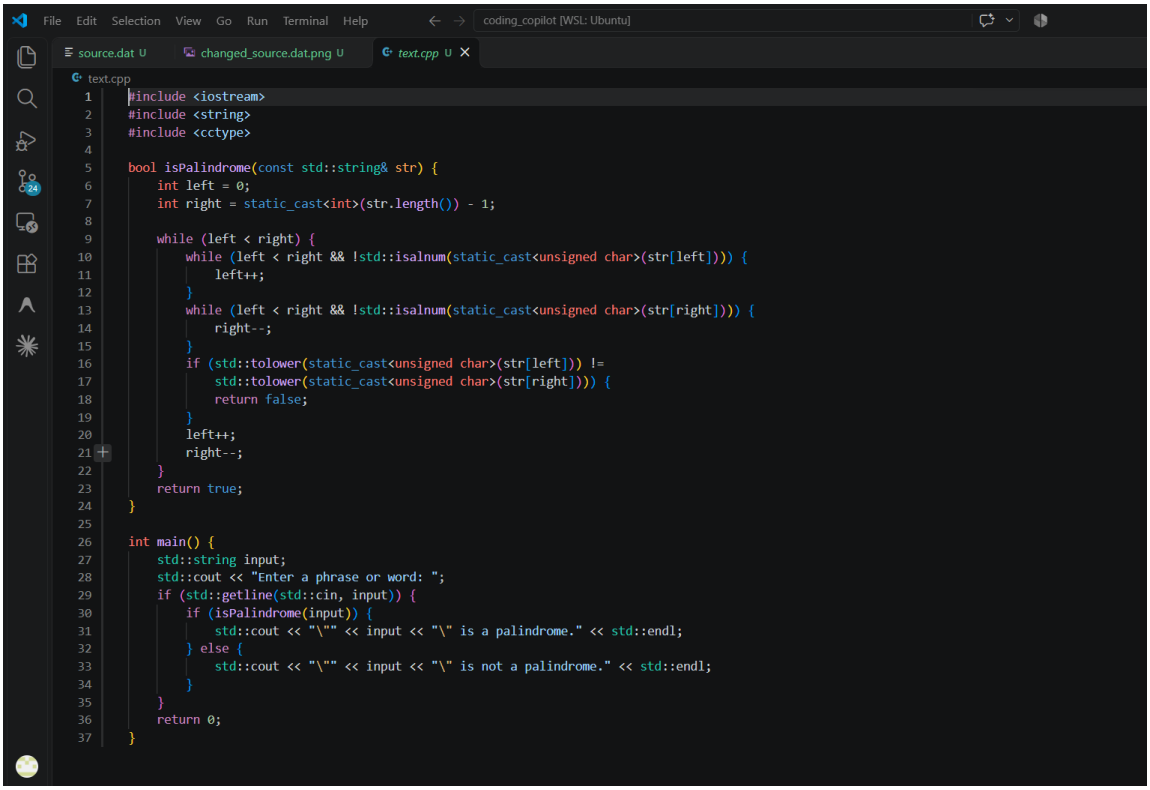
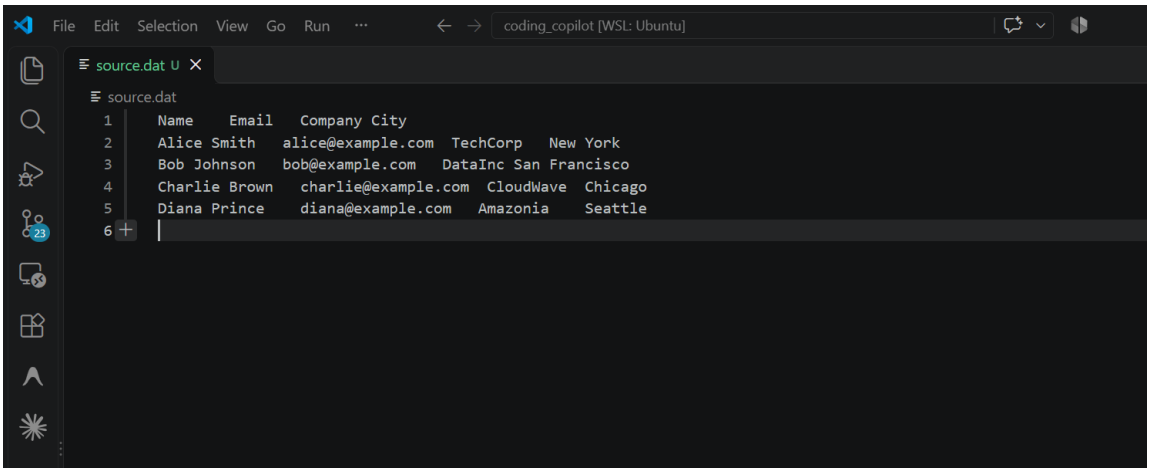


Figure 14.11: CLI refactoring of a non-C++ file (`source.dat`): read permission, unified diff, approval, write permission and overwrite confirmation



(a) `text.cpp` saved by `generate`



(b) `source.dat` after `refactor`: commas became tabs

Figure 14.12: Files written by the CLI, each only after the user's explicit approval

14.13.2 Browser GUI

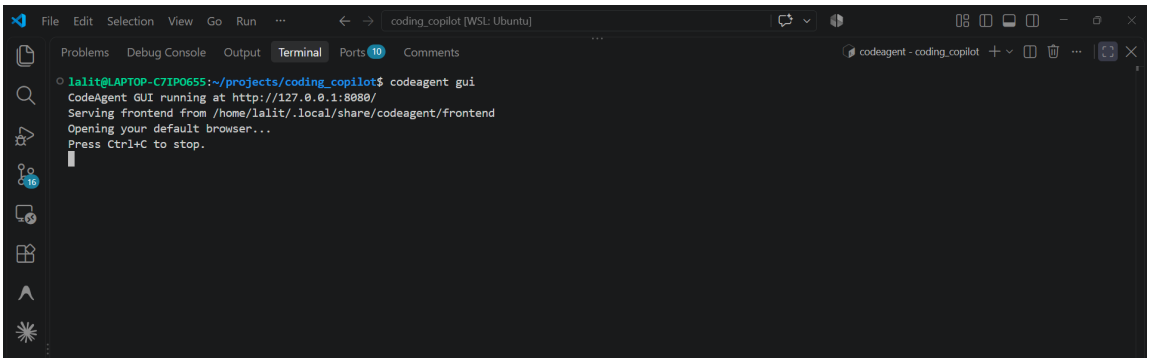


Figure 14.13: Starting the GUI backend with `codeagent gui`

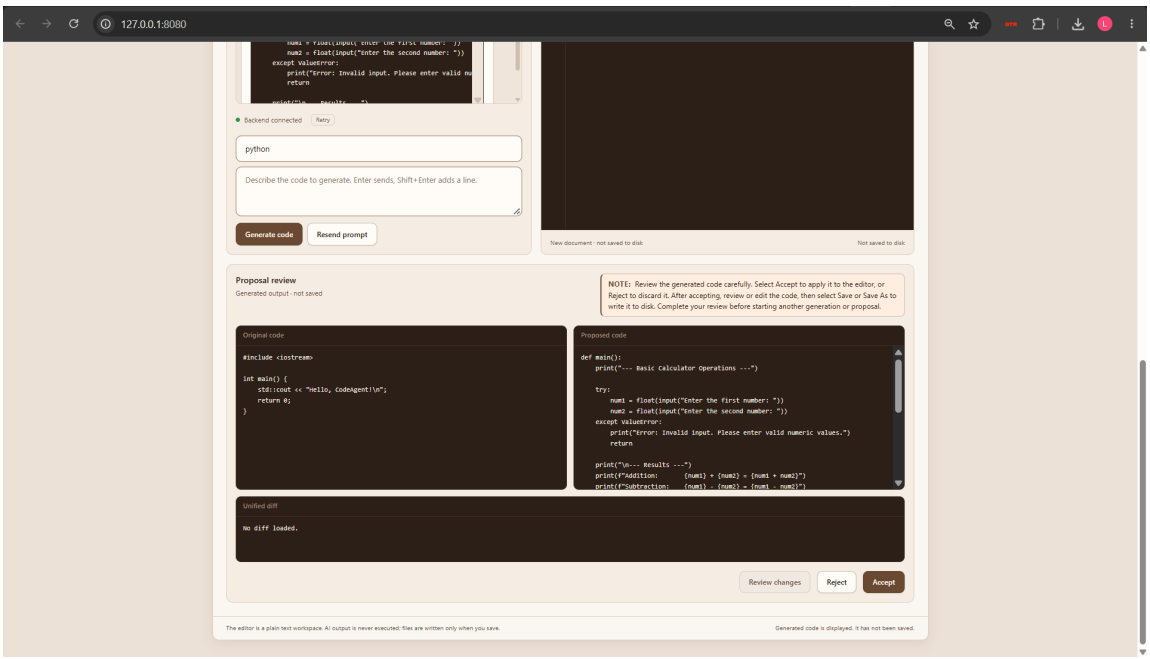


Figure 14.14: GUI code generation in Python with the backend connected: the generated output is shown for review and is not saved

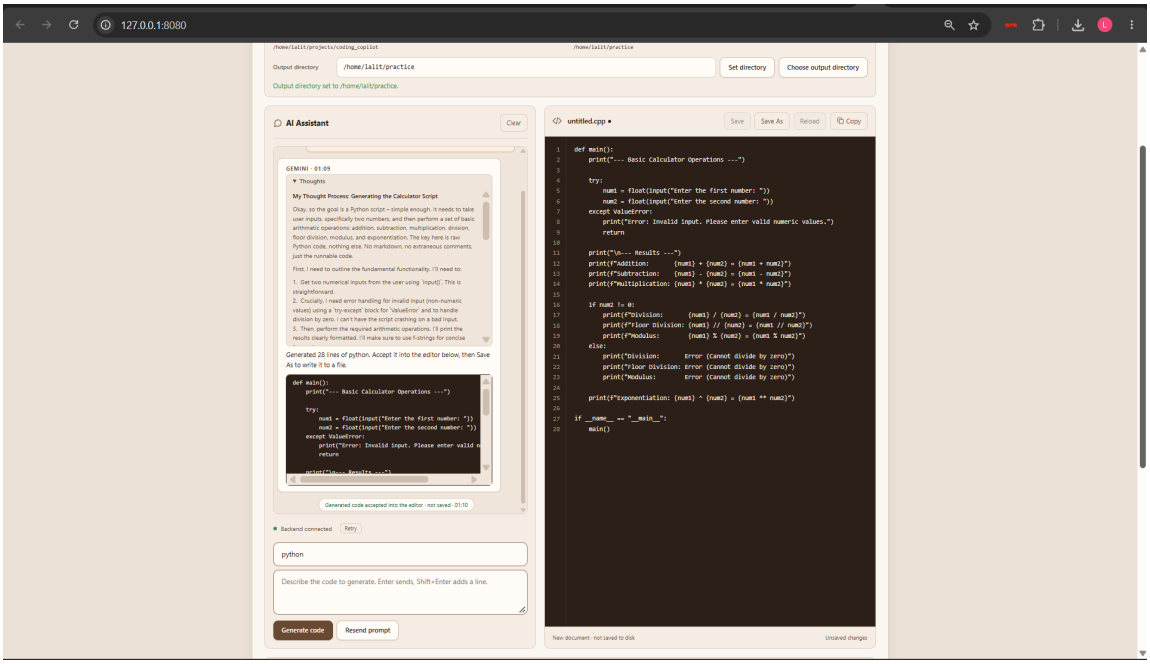


Figure 14.15: Generated code accepted into the editor (not saved), with Gemini's thoughts shown in the chat

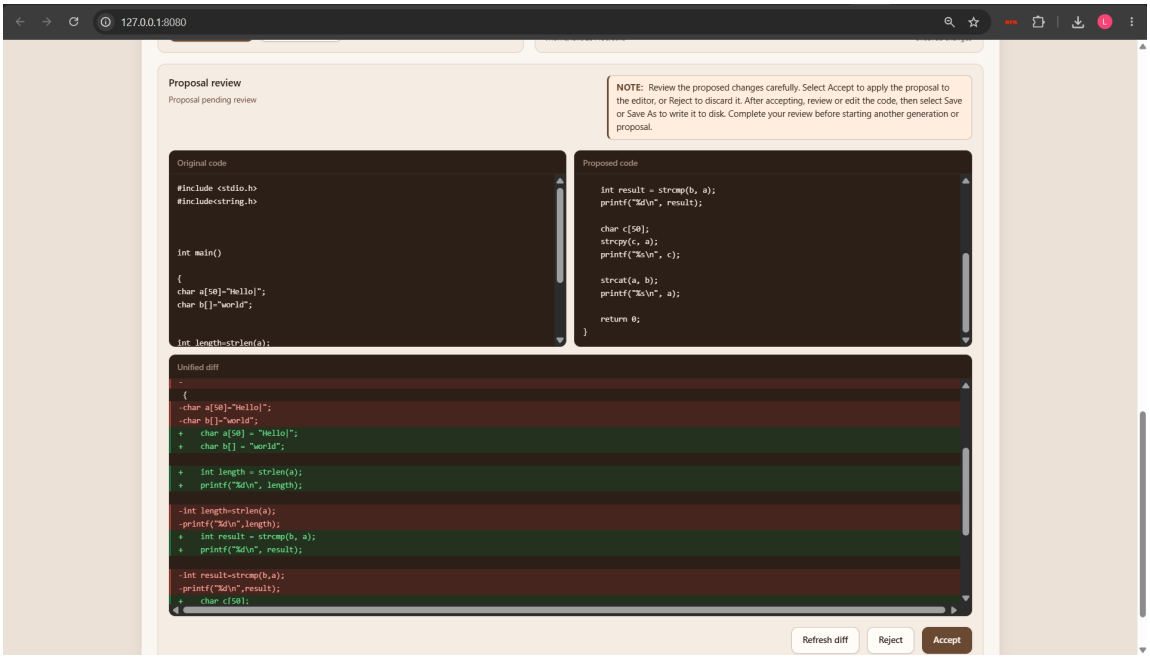


Figure 14.16: GUI refactoring proposal: original code, proposed code and the coloured unified diff, with Accept and Reject

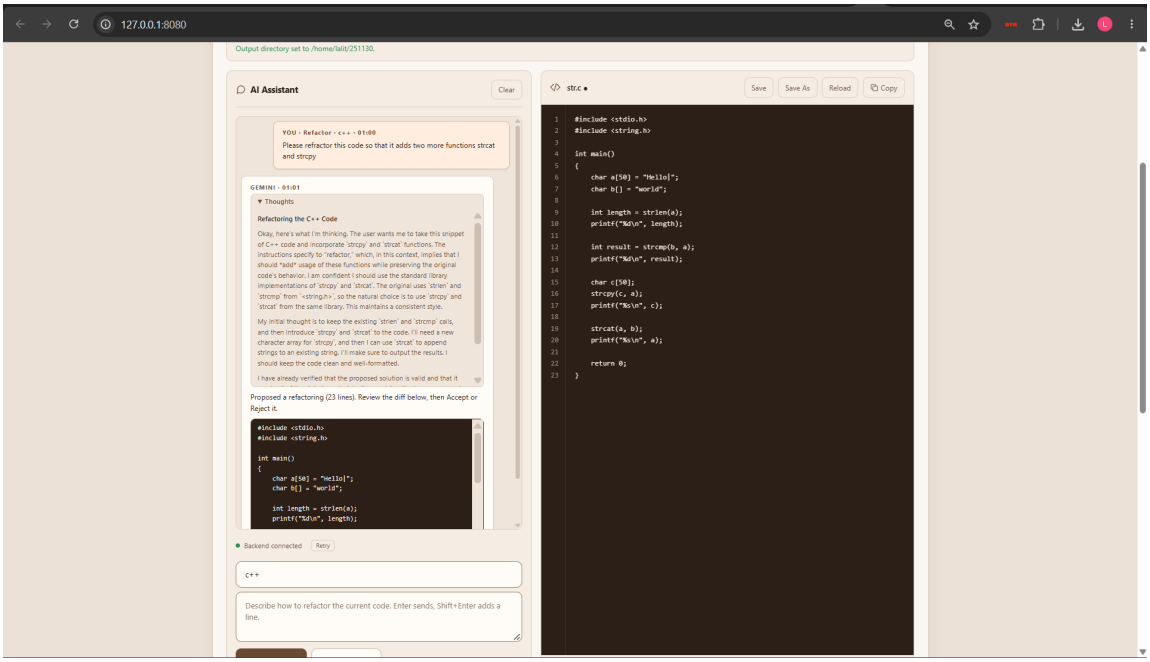


Figure 14.17: Approved proposal applied to the editor; the unsaved marker on `str.c` shows nothing has been written yet

14.14 Demonstration Plan

1. **Build and tests:** `cmake --build build`, `ctest`, `node --test` (Figures 14.7 and 14.8).
2. **CLI generation:** generate a palindrome checker, save it, then try to save it to the same name again to show the overwrite confirmation.
3. **CLI refactoring:** refactor a small file, show the diff, reject it (the file is unchanged), run it again and approve.
4. **Error handling:** unset the API key to show the configuration error and exit code 1.
5. **GUI:** `codeagent gui` → open a file by path → refactor → review the diff → accept (the editor changes, the disk does not) → edit the file in another editor → Save shows the conflict confirmation.
6. **Code walk-through:** `CodeAgentError` polymorphism, `respondWithError`, RAII guards, the dependency-injection test seams, and the pimpl header.

14.15 Limitations and Future Work

- Share the GUI's stronger atomic writer (permission preservation, `fsync`, unique temporary names) with the CLI `FileManager`.
- Give each FileManager test its own sandbox directory so the suite can run in parallel (`ctest -j`).
- Extract a `ProposalStore` class from `GuiHttpServer::Impl` to reduce its size.
- Add a `--language` option to the CLI `generate` command.
- Optionally restrict which paths the GUI may open, beyond the `.env` rule.
- Make the Gemini model configurable, add syntax highlighting, and add an automated real-browser end-to-end test and CI.

14.16 Conclusion

CodeAgent meets its requirements:

- it generates and refactors code with Gemini from both a CLI and a browser GUI (R1, R2, R7);
- it never changes a file without an explicit, separate save decision (R3, R4);
- it keeps the API key out of URLs, logs and the browser (R5);
- it turns every failure into a typed, actionable message (R6);
- it is verified by 293 automated tests that run offline (R8).

The architecture demonstrates object-oriented design in practice: small encapsulated classes, an abstract exception hierarchy with virtual functions, interface-based polymorphism, composition, and dependency injection. On the advanced C++ side it uses RAII and smart pointers, exceptions, templates, functors, concurrency and C++17 features. The Git history shows incremental and parallel work by both members.